

MACHINE LEARNING FUNDAMENTALS SERIES

Sequence-to-Sequence (Seq2Seq) Encoder-Decoder Networks

Variable-Length Processing, Context Vector Mechanics, Teacher Forcing, and Scale Comparison

1. Introduction & Sequence-to-Sequence Challenges

Standard deep learning architectures (such as Feedforward NNs or CNNs) require fixed-size input and output dimensions. However, real-world Natural Language Processing (NLP) tasks—such as Machine Translation, Text Summarization, and Question Answering—face two major structural challenges:

- **Variable-Length Sequences:** Input sentences vary dynamically in length (e.g., "Let's go" [2 words] vs. "My name is Statquatch" [4 words]).
- **Mismatched Input/Output Lengths:** Source and target sequences rarely have a 1-to-1 word alignment. For instance, translating the 2-word English phrase "Let's go" into Spanish yields a 1-word target "Vamos".

The **Sequence-to-Sequence (Seq2Seq)** framework solves this using two specialized recurrent sub-networks: an **Encoder** and a **Decoder**.

2. The Encoder Block: Role & Internal Mechanics

2.1 Primary Objective

The Encoder takes a variable-length source sequence and compresses all semantic and grammatical context into a single, fixed-size numerical tensor called the **Context Vector**.

2.2 Step-by-Step Processing Flow

1. **Word Embedding Layer:** Raw categorical input tokens (e.g., "let's", "go") are converted into dense continuous vectors via a source Word Embedding lookup table.
2. **Unrolled LSTM Processing:** Dense embeddings feed sequentially into unrolled Long Short-Term Memory (LSTM) units across time steps. Crucially, **all time steps reuse the exact same weight matrices and biases** (*Weight Sharing*).
3. **Deep Stacked Layers:** To capture complex hierarchical patterns, the Encoder stacks multiple LSTM layers (e.g., Layer 1 processes word embeddings; Layer 2 processes Layer 1's hidden states).
4. **Context Vector Generation:** The final hidden states (h) and cell states (c) from all LSTM layers at the last input time step are extracted to form the **Context Vector**. This vector serves as the sole information bottleneck connecting source and target languages.

3. The Decoder Block: Role & Internal Mechanics

3.1 Primary Objective

The Decoder decompresses the **Context Vector** to generate the target sequence word-by-word in an autoregressive fashion.

3.2 Autoregressive Generation Workflow

1. **State Initialization:** The hidden and cell states of all LSTM layers in the Decoder are **initialized directly with the Context Vector** provided by the Encoder. (*Note: Decoder LSTMs maintain independent trainable parameters from the Encoder*).

2. **Start Token Input:** Autoregressive generation begins by feeding a start-of-sequence token (such as <EOS> or <SOS>) through the target language Embedding Layer.
3. **Fully Connected & SoftMax Layer:** The output from the top Decoder LSTM layer passes through a Fully Connected projection layer matching the target vocabulary size, followed by a **SoftMax** activation to compute class probabilities.
4. **Autoregressive Feedback Loop:** At inference, the token predicted at time step t is fed back as the input token for step $t+1$. Generation continues iteratively until the Decoder emits the end-of-sequence token (<EOS>) or hits a predefined max length limit.

4. Training Optimization Strategy: Teacher Forcing

During model training, autoregressive generation can cause *exposure bias*—if the Decoder predicts an incorrect word at step t , subsequent steps receive corrupted inputs, causing errors to compound rapidly.

Teacher Forcing Mechanism

To stabilize training, **Teacher Forcing** replaces the model’s unconditioned prediction at step t with the **actual ground truth target token** when feeding into step $t + 1$. This prevents error accumulation, stabilizes gradient flow during Backpropagation, and significantly accelerates model convergence.

5. Comparative Scale Analysis: Demo Model vs. Original Paper

The table below contrasts the educational demonstration architecture with the landmark original paper (*Sutskever et al., 2014: "Sequence to Sequence Learning with Neural Networks"*).

Table 1: Structural and Parameter Scale Comparison

Architecture Metric	Feature	/	StatQuest Demo Model	Original Paper (Sutskever et al.)
Input ($ V_{in} $)	Vocabulary	Size	4 tokens ("let's", "go", <EOS>)	160,000 tokens
Output ($ V_{out} $)	Vocabulary	Size	4 tokens ("ir", "vamos", "y", <EOS>)	80,000 tokens
Word Embedding Dimension	Embedding	Dimen- sion	2 dimensions / token	1,000 dimensions / token
Number of LSTM Layers			2 layers (Encoder & Decoder)	4 stacked layers
LSTM Units per Layer			2 hidden units / layer	1,000 hidden units / layer
Fully Connected Layer	Connected	Output	2 inputs → 4 outputs	1,000 inputs → 80,000 outputs
Total Trainable Parameters			220 weights & biases	384,000,000 (384M) parameters

6. Conclusion

The minimalist 220-parameter demonstration effectively captures every core mathematical and structural principle of the full 384-million parameter Seq2Seq model. By decoupling sequence encoding from target decoding via a compressed Context Vector, Seq2Seq laid the foundational groundwork for modern machine translation systems and paved the way for attention mechanisms and Transformer architectures.

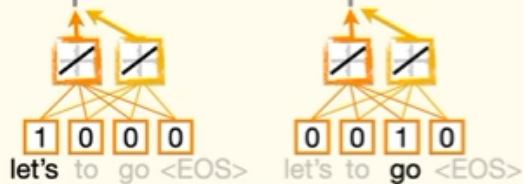


NOTE: Just like for all **Neural Networks**, all of these **Weights** and **Biases** are trained using **Backpropagation**.

Encoder

Layer 2

Layer 1



Decoder

